

Professional Coding Specialist

COS Pro 파이썬 2 급

10 강- 13 강. 모의고사 2 차

모의고사 2 차 문제 1-10 번

과정 소개

모의고사(YBM 샘플) 문제를 통해 실전에 대비할 수 있다. (회차당 10 문항, 각 문항별 문제 풀이)

학습 목차

모의고사 2 차 문제 1 번
모의고사 2 차 문제 2 번
모의고사 2 차 문제 3 번
모의고사 2 차 문제 4 번
모의고사 2 차 문제 5 번
모의고사 2 차 문제 6 번
모의고사 2 차 문제 7 번
모의고사 2 차 문제 8 번
모의고사 2 차 문제 9 번
모의고사 2 차 문제 10 번

학습 목표

1. YBM IT(www.ybmit.com) 에서 제공하는 COS Pro 파이썬 2 급 샘플 문제 메뉴의 모의고사 문제를 풀어보며 COS Pro 파이썬 2 급 시험에 대비할 수 있다.

1

모의고사 2 차 문제 1 번

문제 1 의 정답

제품 번호별 장갑 짝 맞추기

```

max_product_number = 10

def func_a(arr):
    max_product_number = 10
    counter = [0 for _ in range(max_product_number + 1)]
    for x in arr:
        counter[x] += 1
    return counter

def solution(left_gloves, right_gloves):
    left_counter = func_a(left_gloves)
    right_counter = func_a(right_gloves)

    total = 0
    for i in range(1, max_product_number + 1):
        total += min(left_counter[i], right_counter[i])
    return total

```

왼쪽 장갑 개수

0	1	2	3	4	5	6	7	...	10	제품번호 (인덱스번호)
0	1	3	0	1	0	0	0			장갑개수

오른쪽 장갑 개수

0	1	2	3	4	5	6	7	...	10
0	1	2	0	2	0	0	1		

1. 왼손 장갑이 제품 번호별로 몇 개씩 있는지 개수를 셉니다.
2. 오른손 장갑이 제품 번호별로 몇 개씩 있는지 개수를 셉니다.
3. 각 제품 번호별로 최대한 많은 장갑 쌍을 만들면서 개수를 셉니다.

- 표현식을 사용하여 11 개의 0 을 항목으로 갖는 리스트를 만든다.
- for 문으로 arr 리스트의 데이터를 x 변수에 순서대로 가져와서 x 의 값과 같은 인덱스 번호에 해당하는 값을 증가시킨다.

```
max_product_number = 10

def func_a(arr):
    max_product_number = 10
    counter = [0 for _ in range(max_product_number + 1)]
    for x in arr:
        counter[x] += 1
    return counter
```

- 왼손 장갑의 제품 번호별 개수를 left_counter 리스트에 넣고, 오른쪽 장갑의 제품 번호별 개수를 right_counter 변수에 넣는다.

```
def solution(left_gloves, right_gloves):
    left_counter = func_a(left_gloves)
    right_counter = func_a(right_gloves)

    total = 0
    for i in range(1, max_product_number + 1):
        total += min(left_counter[i], right_counter[i])
    return total
```

- total 변수를 0 으로 초기화 한다.
- for 문을 이용하여 인덱스 번호 1 번부터 10 번까지 돌면서 왼손 장갑과 오른손 장갑 번호가 같은 것(즉 인덱스 번호가 같은 것)끼리 비교하여 더 작은 수를 total 변수에 더하면 만들 수 있는 장갑 쌍의 개수를 구한다.

문제 2 의 정답

3 의 배수와 5 의 배수 중 더 많은 것

```

def func_a(arr):
    count = 0
    for n in arr:
        if n % 5 == 0:
            count += 1
    return count

def func_b(three, five):
    if three > five:
        return "three"
    elif three < five:
        return "five"
    else:
        return "same"

def func_c(arr):
    count = 0
    for n in arr:
        if n % 3 == 0:
            count += 1
    return count

def solution(arr):
    count_three = func_c(arr)
    count_five = func_a(arr)
    answer = func_b(count_three, count_five)
    return answer

```

< func_a 함수 >

- count 를 0 으로 초기화한다.
- arr 리스트의 데이터를 하나씩 n 에 넣어서 n 을 5 로 나눈 나머지가 0 이면, 즉, n 이 5 의 배수이면 count 를 증가시킴으로써 arr 리스트에 들어 있는 5 의 배수인 수의 개수를 센다.

```

def func_a(arr):
    count = 0
    for n in arr:
        if n % 5 == 0:
            count += 1
    return count

```

< func_b 함수 >

- 3 의 배수의 개수를 매개변수 three 에, 5 의 배수의 개수를 five 매개변수에 받아서
- three 변수가 더 크면 "three"를
- five 변수가 더 크면 "five"를
- 같으면 "same" 이라는 문자열을 리턴한다.

```
def func_b(three, five):  
    if three > five:  
        return "three"  
    elif three < five:  
        return "five"  
    else:  
        return "same"
```

< func_c 함수 >

- count 를 0 으로 초기화한다.
- arr 리스트의 데이터를 하나씩 n 에 넣어서 n 을 3 으로 나눈 나머지가 0 이면, 즉, n 이 3 의 배수이면 count 를 증가시킴으로써 arr 리스트에 들어 있는 3 의 배수인 수의 개수를 센다.

```
def func_c(arr):  
    count = 0  
    for n in arr:  
        if n % 3 == 0:  
            count += 1  
    return count
```

3

모의고사 2 차 문제 3 번

문제 3 의 정답

N 부터 M 까지 자연수 중 짝수들의 제곱의 합

```
def solution(N, M):  
    total = 0  
    for x in range(N, M + 1):  
        if x % 2 == 0:  
            total += x*x  
    return total
```

4 부터 7 까지의 자연수 중에서 짝수는 4 와 6 입니다. $4^2 + 6^2 = 16 + 36 = 52$

- ♦ total 변수를 0 으로 초기화합니다.
- ♦ for 문을 이용하여 N 부터 M 까지 돌면서 자연수를 하나씩 x 에 꺼냅니다.
- ♦ x 의 값을 2 로 나눈 나머지가 0 이면, 즉 x 가 짝수이면 total 변수에 x 의 제곱(x*x)을 더합니다.

```
def solution(N, M):  
    total = 0  
    for x in range(N, M + 1):  
        if x % 2 == 0:  
            total += x*x  
    return total
```

※ 참고 : 제곱 표현식

** 연산자

pow() 함수

예시) 2 의 세제곱 : 2**3 / pow(2,3)

4

모의고사 2 차 문제 4 번

문제 4 의 정답

길이가 5 이상인 단어 붙이기

```
def solution(words):  
    answer = ""  
  
    for w in words:  
        if len(w) >= 5:  
            answer += w  
  
    if len(answer) < 1:  
        answer = "empty"  
    return answer
```

단어들이 들어 있는 리스트에서 길이가 5 이상인 단어를 리스트에 들어 있는 순서대로 이어 붙이려고 합니다.

만약 return 할 문자열이 빈 문자열이면 "empty"를 return 해 주세요.

words	return
["my", "favorite", "color", "is", "violet"]	"favoritecolorviolet"
["yes", "i", "am"]	"empty"

- words 리스트에 들어 있는 단어를 하나씩 w 변수에 읽어 들어서 w의 길이가 5 이상이면 해당 단어를 answer 변수에 순서대로 붙여준다.
- answer 변수에 들어 있는 값이 없으면 (answer의 길이가 1 이하이면) answer에 "empty" 글자를 넣어 empty를 리턴한다.

```
def solution(words):
    answer = ""

    for w in words:
        if len(w) >= 5:
            answer += w

    if len(answer) < 1:
        answer = "empty"
    return answer
```

5

모의고사 2 차 문제 5 번

문제 5의 정답

최소 공격 횟수

```
def solution(attack, recovery, hp):
    count = 0

    while(True):
        count += 1
        hp -= attack
        if hp <= 0:
            break
        hp += recovery
```

몬스터는 처음에 일정 수치의 체력(HP)을 가지고 있고,
캐릭터가 전투에서 이기려면 몬스터를 공격해 몬스터의 체력을 0 이하로 만들어야 합니다.
캐릭터는 공격 마법만 사용하며, 공격 마법은 항상 같은 데미지를 입힙니다.
몬스터는 힐링 마법만을 사용하며, 힐링 마법은 항상 같은 수치로 체력을 회복합니다.
둘은 항상 번갈아 가며 마법을 사용하고, 처음에는 항상 캐릭터가 먼저 공격을 시작합니다.

attack	recovery	hp	return
30	10	60	3

차례	몬스터 hp 변화	몬스터 남은 hp
		60
캐릭터	공격 -30	30
몬스터	회복 +10	40
캐릭터	공격 -30	10
몬스터	회복 +10	20
캐릭터	공격 -30	-10

- 캐릭터가 공격한 횟수를 증가시킵니다.
- 캐릭터를 공격하면 몬스터의 체력(hp)은 공격(attack)만큼 감소해야 하므로 [체력(hp) - 공격(attack)]한 값을 다시 체력(hp)에 넣어줍니다.
- 몬스터의 체력(hp)이 0 이하이면 게임을 종료해야 하므로 while 반복문을 빠져나가기 위해 break 문장을 사용합니다.
- 몬스터의 체력(hp)이 0 보다 크면 게임을 계속해야 하고, 캐릭터가 공격하고 나면 몬스터가 힐링 마법을 사용할 차례입니다. (캐릭터와 몬스터는 서로 번갈아 공격 혹은 회복을 하기 때문)
따라서 몬스터는 힐링마법으로 몬스터는 체력을 회복하므로 [체력(hp) + 회복(recovery)]을 체력(hp)에 넣어 줍니다.

```
def solution(attack, recovery, hp):
    count = 0

    while(True):
        count += 1
        hp -= attack
        if hp <= 0:
            break
        hp += recovery
```

6

모의고사 2 차 문제 6 번

문제 6 의 정답

엘리베이터 총 이동 거리

```
def solution(floors):
    dist = 0
    length = len(floors)

    for i in range(1, length):
        if floors[i] > floors[i-1]:
            dist += floors[i] - floors[i-1]
        else:
            dist += floors[i-1] - floors[i]

    return dist
```

하루 동안 엘리베이터가 멈춘 층이 순서대로 들어 있는 리스트가 있습니다.

이때, 엘리베이터의 총 이동거리를 구하려고 합니다.

단, 층과 층 사이의 거리는 1 입니다.

floors	return
[1, 2, 5, 4, 2]	7

1 3 1 2

- 인덱스 번호를 1 부터 리스트의 총 길이만큼 움직이며 데이터를 읽어 들인다. (0 번부터 시작하면 바로 앞의 인덱스 번호와 비교할 수 없으므로 주의)
- 읽어 들인 데이터의 바로 앞 인덱스에 있는 값과 현재 인덱스에 있는 값의 차이를 구해서 dist 변수에 합한다.
- 이때 앞의 값과 뒤의 값의 차가 절대값이어야 하므로 이전 값과 현재 값 중 큰 값을 찾아서 큰 수에서 작은 수를 빼기 위해 if 문을 사용한다.

```
def solution(floors):
    dist = 0
    length = len(floors)

    for i in range(1, length):
        if floors[i] > floors[i-1]:
            dist += floors[i] - floors[i-1]
        else:
            dist += floors[i-1] - floors[i]

    return dist
```

다른 코딩 제안 : abs() 함수 -> 절대값을 구하는 함수

```
def solution(floors):
    dist = 0
    length = len(floors)
    for i in range(1,length):
        dist += abs(floors[i] - floors[i-1])
    return dist
```

7

모의고사 2 차 문제 7 번

문제 7 의 정답

```
def solution(value, unit):
    converted = 0
    if unit == "C":
        value = (value * 1.8) + 32
    if unit == "F":
        value = (value - 32) / 1.8
    converted = int(value)
    return converted
```

- * 화씨온도(°F)에서 섭씨온도(°C)로 환산 : (화씨온도 - 32) ÷ 1.8 → 섭씨온도
- * 섭씨온도(°C)에서 화씨온도(°F)로 환산 : (섭씨온도 × 1.8) + 32 → 화씨온도

value	unit	return
527	"C"	980

- uint 이 C 이면 화씨로 변환하고, unit 이 F 이면 섭씨로 변환한다.
- 연산자 우선 순위 문제로 곱하기, 나누기가 더하기, 빼기 보다 우선 순위임을 주의한다. value-32 를 먼저 하기 위해 (value-32)와 같이 괄호를 해 주어야 한다.
- 정수 부분만 리턴하기 위해 int() 함수를 사용한다.

```
def solution(value, unit):
    converted = 0
    if unit == "C":
        value = (value * 1.8) + 32
    if unit == "F":
        value = (value - 32) / 1.8
    converted = int(value)
    return converted
```

※시험장에서는 다른 샘플 데이터를 테스트 할 수 없으나, 연습 시에는 아래와 같은 데이터도 잘 되는지 확인해 본다.

```
value = 600
unit = "F"
ret = solution(value, unit)
```

8

모의고사 2 차 문제 8 번

문제 8 의 정답

소수의 개수

```
def solution(number):
    count = 0
    while number:
        n = number % 10
        if n == 2 or n == 3 or n == 5 or n == 7:
            count += 1
        number //= 10
    return count
```

자연수를 각 자릿수별로 나누었을 때 2, 3, 5, 7 의 개수 세기

number	return
29022531	5

2 2 2 5 3

맨 뒷자리부터 숫자를 하나씩 꺼내서
2, 3, 5, 7 중 하나이면 개수 증가

- 맨 마지막 자리부터 맨 앞자리까지 숫자 중 2, 3, 5, 7 의 개수 세기 위한 코드이다.
- number 변수가 0 이 아닌 동안 반복하기 위해 number 를 사용한다. number 대신 number!=0 혹은 number>0 로 사용해도 된다.
 - 현재 숫자의 마지막 자리를 찾기 위해 number 을 10 으로 나눈 나머지(number%10)를 사용하여 n 변수에 넣는다.
 - 맨 마지막 자리 수인 n 이 2, 3, 5, 7 중 하나이면 count 변수에 개수를 증가시킨다.
 - 현재 숫자에서 맨 뒷자리를 제거하기 위해 number 를 10 으로 나눈 몫(number//10)을 다시 number 에 집어넣는다.

```
def solution(number):
    count = 0
    while number:
        n = number % 10
        if n == 2 or n == 3 or n == 5 or n == 7:
            count += 1
        number //= 10
    return count
```

9

모의고사 2 차 문제 9 번

문제 9 의 정답

K 표를 받은 후보의 수

```
def solution(votes, N, K):
    counter = [0 for _ in range(N + 1)]
    for x in votes:
        counter[x] += 1
    answer = 0
    for c in counter:
        if c == K:
            answer += 1
    return answer
```

5 명의 후보에 대해 투표를 진행한 결과 [2, 5, 3, 4, 1, 5, 1, 5, 5, 3]
 [2 번, 5 번, 3 번, 4 번, 1 번, 5 번, 1 번, 5 번, 5 번, 3 번] 후보에 투표
 정확히 K 표를 받은 후보는 총 몇 명?

투표 결과		후보수 표의 개수 결과값		
votes		N	K	return
[2, 5, 3, 4, 1, 5, 1, 5, 5, 3]		5	2	2

counter						
0	1	2	3	4	5	→ 후보 번호
0	2	1	2	1	4	→ 후보 번호별 득표수

K : 2

2 표를 받은 후보 : 1 번, 3 번 → 2 명

- ♦ counter 리스트를 0 으로 초기화한다.

- ♦ votes 리스트의 데이터를 읽어 x 변수에 넣고, counter 리스트의 x 번 인덱스에 투표 수(개수)를 증가한다.

- ♦ counter 리스트에 들어 있는 값 중 K 와 같은 값을 가진 후보의 인원수를 구하기 위한 answer 변수를 0 으로 초기화 한다. (문제에서는 -1 로 초기화했기 때문에 0으로 변경)

- ♦ counter 리스트에 들어 있는 값 중 K 와 같은 값을 가진 경우 answer 를 증가시킨다.

```
def solution(votes, N, K):  
    counter = [0 for _ in range(N + 1)]  
    for x in votes:  
        counter[x] += 1  
    answer = 0  
    for c in counter:  
        if c == K:  
            answer += 1  
    return answer
```

10

모의고사 2 차 문제 10 번

문제 10 의 정답

구매 금액에 따른 상품권 지급

```
def solution(purchase)
    total = 0
    for p in purchase:
        if p >= 1000000:
            total += 50000
        elif p >= 600000:
            total += 30000
        elif p >= 400000:
            total += 20000
        elif p >= 200000:
            total += 10000
    return total
```

지급 가능한 가장 큰 금액으로 상품권 1 장만 지급, 지급해야 할 상품권 총액 구하기

구매 금액	상품권
100만원 이상 구매	5만원 상품권
60만원 이상 구매	3만원 상품권
40만원 이상 구매	2만원 상품권
20만원 이상 구매	1만원 상품권

purchase	return
[150000, 210000, 399990, 990000, 1000000]	100000

- ♦ 210,000원, 399,990원을 구매한 고객에게 1만원 상품권 지급 → 총 2만원
- ♦ 990,000원을 구매한 고객에게 3만원 상품권 지급 → 총 3만원
- ♦ 1,000,000원을 구매한 고객에게 5만원 상품권 지급 → 총 5만원

- ♦ 지급해야 할 상품권 총액을 저장할 total 변수를 0 으로 초기화한다.
- ♦ purchase 리스트에 있는 데이터를 가져와서 p 에 넣은 후 상품권 지급표에 있는 구매 금액에 맞게 총 지급해야 할 상품권 금액(total)을 더한다.
- ♦ 마지막에 20 만원 이상이면 상품권 1 만원을 지급해야 하므로 else 대신 elif 를 사용하여 p>=200000 으로 수정한다.

```
def solution(purchase)

    total = 0

    for p in purchase:
        if p >= 1000000:
            total += 50000
        elif p >= 600000:
            total += 30000
        elif p >= 400000:
            total += 20000
        elif p >= 200000:
            total += 10000

    return total
```